



End-to-End Data Pipeline Deployment for Food Delivery Industry on Cloud (GCP)

ENGR - 516: Engineering Cloud Computing
Prof. Dingwen Tao

Team Members:

1. Bhargavi Jahagirdar (bjahagir@iu.edu)
2. Sakshi Gatyani (sgatyan@iu.edu)
3. Arpita Lonakadi (arlona@iu.edu)

1. Project Goals

In today's digital era, the food delivery industry produces vast data, vital for operational optimization and competitiveness. Traditional methods struggle with the scale and complexity of this data. Cloud platforms offer scalable solutions, enabling rapid processing without hefty infrastructure investments. Real-time data analysis enhances decision-making, improves customer experience, and optimise delivery routes. This project proposes an end-to-end ETL pipeline for online food delivery, leveraging parallel processing for optimised handling of large volumes of data and empowering data-driven decision-making. Meeting the escalating demand for scalable big data processing solutions in the food delivery sector, it aims to produce insights in food supply estimation and trends across geographies.

Develop ETL Pipeline:

Create a comprehensive pipeline tailored for online food delivery, encompassing data extraction, transformation, and loading processes to optimise data flow and integrity.

Utilise GCP for Data Management:

Harness the power of Google Cloud Platform to efficiently manage large-scale transactional data, leveraging its scalability, reliability, and security features for seamless operations.

Implement Real-Time Data Processing:

Ingest and transform real-time data from food delivery operations using parallel processing techniques, ensuring timely insights and scalability.

Extract Valuable Insights for Decision-Making:

Enable real-time data processing to drive operational improvements and customer satisfaction, extracting actionable insights for informed decision-making and sustainable growth.

2. Related Work and Gap Analysis

Opportunities in Food Industry Data Analysis:

The food industry's digitalization and rise in online delivery generate vast data, providing predictive insights into trends and hazards. This data is crucial for strategic franchising, emphasising the importance of data-driven decision-making for operational optimization and growth[1].

Challenges in Data Utilisation:

Data volume and standardisation inconsistencies hinder efficient utilisation. Lack of standardised protocols and immature data collection pose challenges in interpreting and sharing data, especially concerning food safety. Cloud computing for food data faces issues like scalability and security, while blockchain's potential remains limited to traceability. Enhanced supply-chain decision-making, food franchising decisions are vital and thus benefited from cloud[2]. Inefficient processing limits actionable insights for decision-making, posing significant challenges for the food industry's effective data utilisation and optimization.

Technological Solutions for Data Challenges:

Cloud computing and enhanced processing techniques offer scalable solutions to address data challenges. Integrating traditional and innovative methods optimises data processing, facilitating informed decision-making and operational excellence in the dynamic food industry landscape.[3] Leveraging cloud computing, constructive ETL[4] and large-scale data handling can address challenges like data volume and inconsistencies in food data standards. Traditional methods like Python ETL and MapReduce, as well as newer approaches like Apache Spark and Apache Beam, optimize data processing with parallel processing. This promises to bridge gaps and enhance scalability in the food industry.

3. Proposed Tasks

Here are the **detailed timelines** for the project: [Project Plan-Timelines and Work Allocation](#)

Week #	Tasks	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
1	Requirement Gathering and Analysis										
2	Architecture Design										
	Data Model Design										
3,4,5	Infrastructure Setup										
	POC: Data Extraction										
	POC: Data Transformation										
	POC: Data Loading										
	Data Extraction										
	Data Transformation										
	Data Loading										
6	Orchestration and Scheduling										
	Data Warehouse Management										
7	Report Generation and Visualization										
8	Testing and Quality Assurance										
9	Deployment										
10	Performance Optimization										

Figure 1: Proposed tasks and timelines

4. Progress on Proposed Tasks

Detailed Progress can be found here: [Project Plan-Timelines and Work Allocation](#)

Week #	Tasks	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
1	Requirement Gathering and Analysis										
2	Architecture Design										
	Data Model Design										
3,4,5	Infrastructure Setup										
	POC: Data Extraction										
	POC: Data Transformation										
	POC: Data Loading										
	Data Extraction										
	Data Transformation										
	Data Loading										
6	Orchestration and Scheduling										
	Data Warehouse Management										
7	Report Generation and Visualization										
8	Testing and Quality Assurance										
9	Deployment										
10	Performance Optimization										

Figure 2: Progress on Proposed tasks and timelines

5. Progress on Proposed Tasks

5.1 Data Description:

We've expanded a Kaggle dataset, adding more data to mirror industry volumes. Table 1 shows the data dictionary. With meticulous enhancements, the dataset now spans gigabytes, supporting robust Big Data pipelines.

Sr no.	Column name	Data Type	Description
1	customer_id	Text	This column stores the unique identifier for each customer. It's of data type Text, indicating that it holds alphanumeric characters
2	date	Date	This column stores the date of the order. It's of data type Date, indicating that it holds date values
3	time	Time	This column stores the time of the order. It's of data type Time, indicating that it holds time values
4	order_id	Text	This column stores the unique identifier for each order. It's of data type Text, indicating that it holds alphanumeric characters
5	items	Text	This column stores the details of the items ordered
6	amount	Integer	This column stores the total amount of the order. It's of data type Integer, indicating that it holds whole number values
7	mode	Text	This column stores the mode of payment for the order
8	restaurant	Text	This column stores the name of the restaurant from which the order was placed
9	Status	Text	This column stores the status of the order
10	ratings	Integer	This column stores the ratings given for the order. It's of data type Integer, indicating that it holds whole number values
11	feedback	Text	This column stores the feedback provided for the order
12	delivery_time	Integer	This column stores the delivery time for the order. It's of data type Integer, indicating that it holds whole number values. The values are in minutes
13	order_preparation_time	Integer	This column stores the time taken to prepare the order. It's of data type Integer, indicating that it holds whole number values. The values are in minutes
14	premium_member	Bit	This column stores information about whether the customer is a premium member or not. It's of data type Bit, indicating that it holds binary data (0 or 1)

15	restaurant_location	Text	This column stores information about the location of the restaurant
----	---------------------	------	---

Table 1: Data Dictionary

5.2 Architecture diagram/ data flow diagram

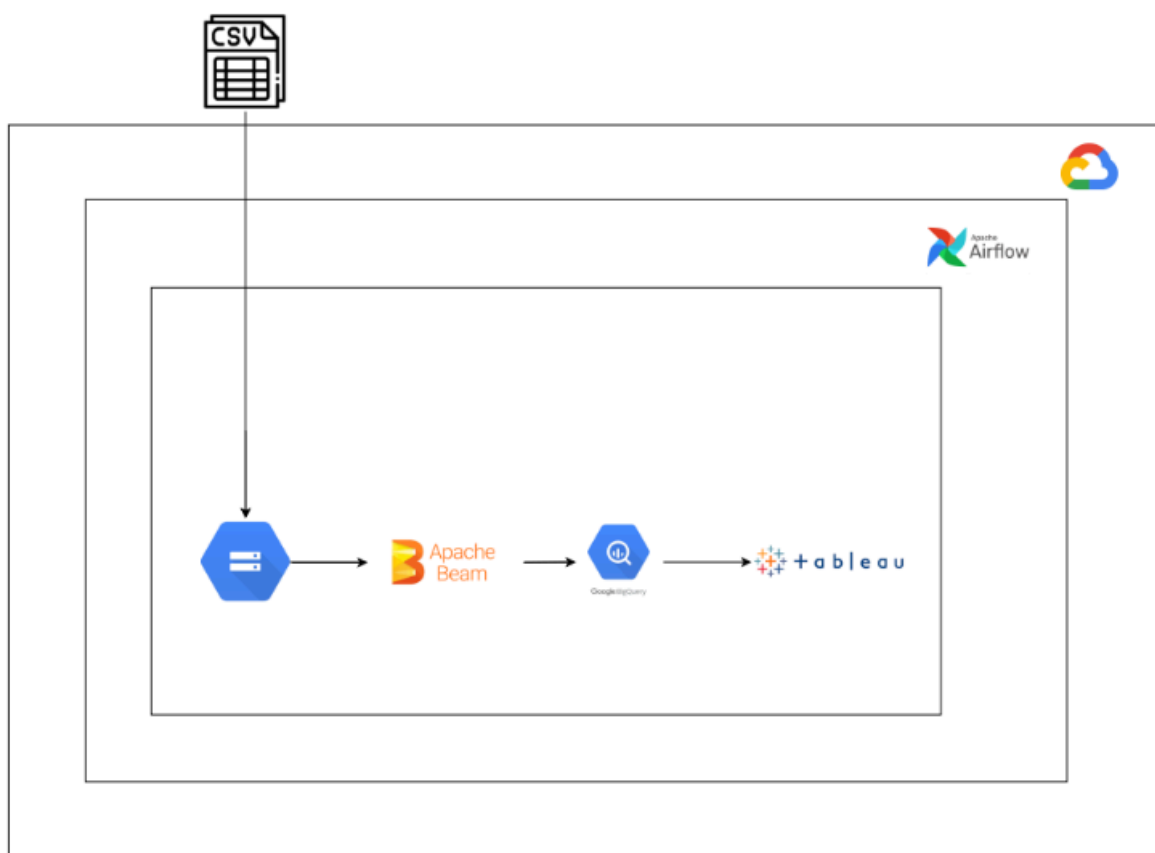


Figure 3: Architecture Diagram

Description of architecture diagram:

1. Google Cloud Storage (GCS) is utilised for storing CSV files, providing a reliable and scalable data storage solution
2. Apache Airflow is employed for orchestrating ETL jobs, ensuring efficient and monitored workflow execution
3. Apache Beam is tasked with data processing, transformation, and loading processed data into Google BigQuery
4. Google BigQuery is a data warehousing solution known for its scalability and real-time analytics capabilities.
5. Tableau serves as our BI tool, leveraging data from Google BigQuery to generate insightful reports and visualisations.

5.3 Data model on BigQuery

Here are the tables that created on BigQuery warehouse:

1. We created a dataset in BigQuery named: food_orders_dataset
2. We are adding two tables: delivered_orders and other_status_orders tables to this dataset
3. The schema for both the tables is as follows:

delivered_orders:

delivered_orders	
row_id	INT
customer_id	Text
date	Date
timestamp	Timestamp
order_id	Text
items	Text
amount	INT
delivery_time	INT
order_preparation_time	INT
premium_member	BIT
feedback	Text
location	Text

Figure 4: Table schema for delivered_orders table

other_status_orders:

other_status_orders	
row_id	INT
customer_id	Text
date	Date
timestamp	Timestamp
order_id	Text
items	Text
amount	INT
order_preparation_time	INT
premium_member	BIT
feedback	Text
location	Text

Figure 5: Table schema for other_status_orders table

5.4 Details on infrastructure setup

1. Project creation on Google Cloud Platform

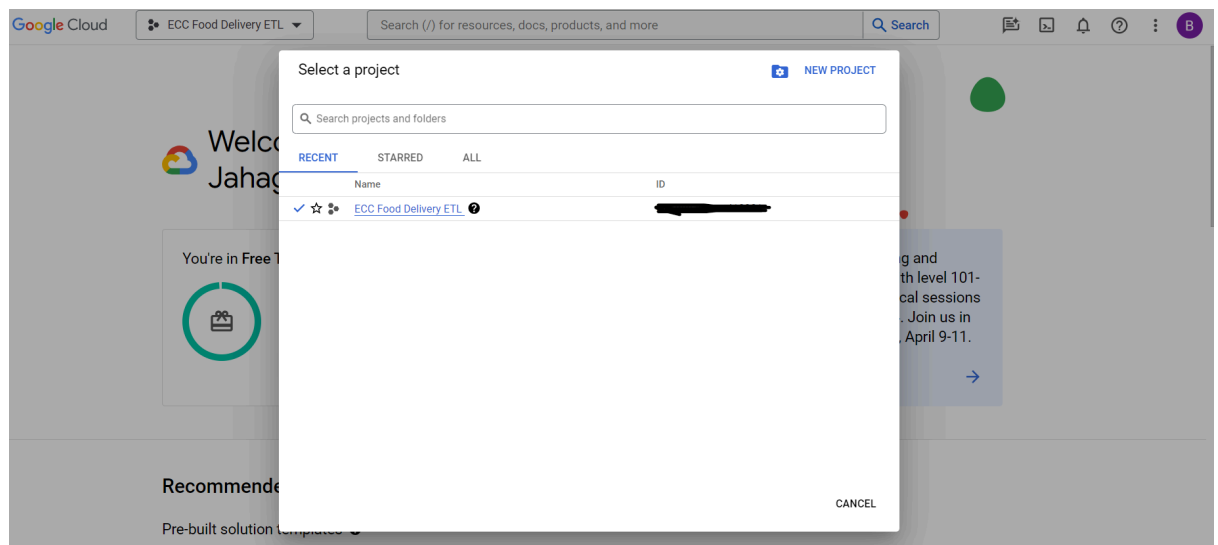


Figure 6: Cloud Project (hiding the ID of the project for security measures)

2. IAM

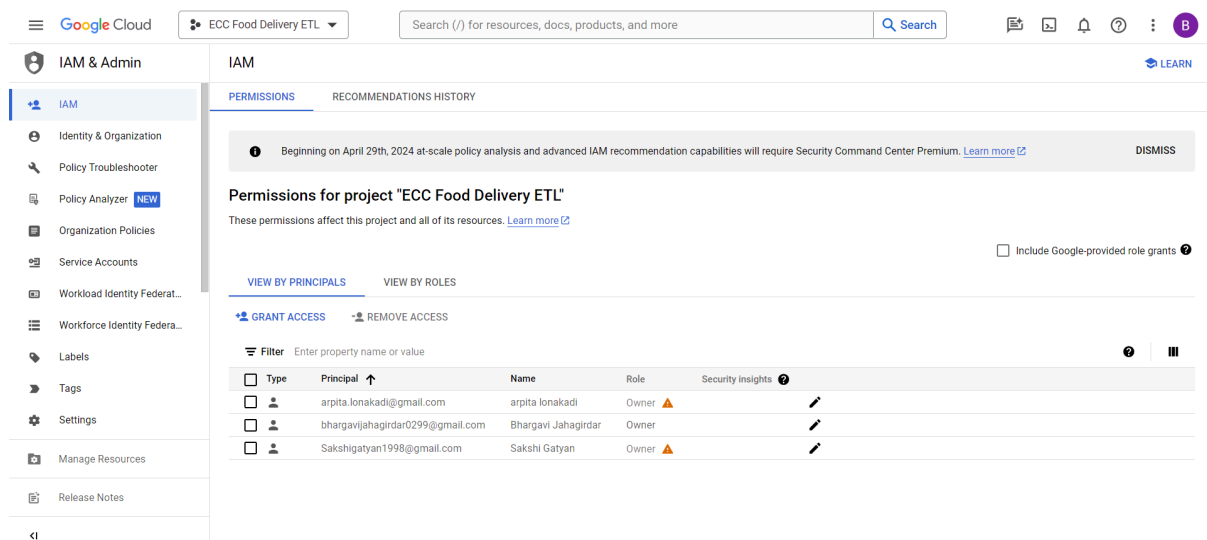


Figure 7: Setting up IAM Access

3. Resource allocation:

- GCS
- BigQuery tables

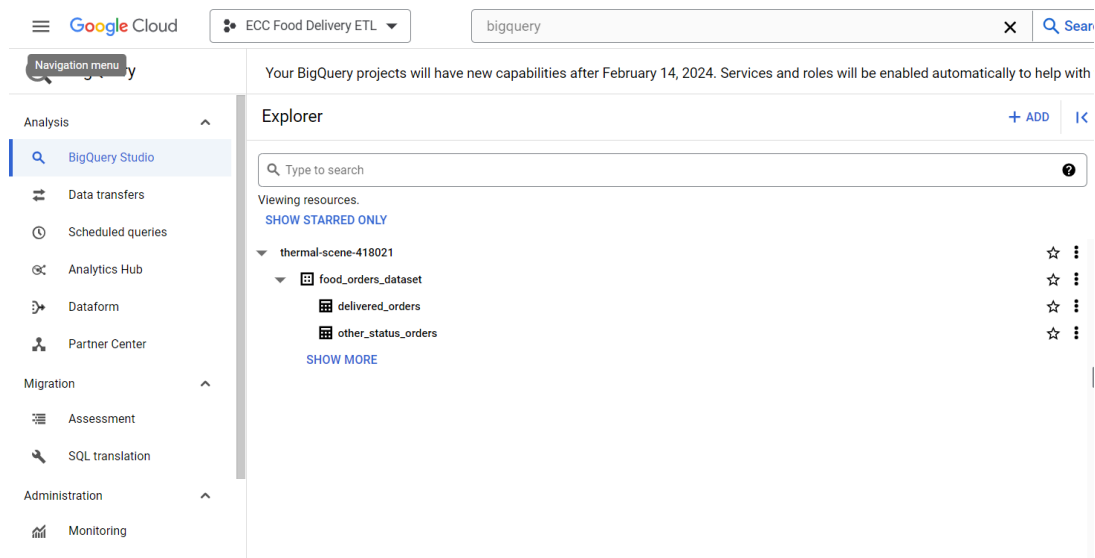


Figure 8: Tables created on BigQuery

The screenshot shows the Google Cloud BigQuery interface. The left sidebar contains navigation options like Analysis, Data transfers, Scheduled queries, Analytics Hub, Dataform, Partner Center, Migration, Assessment, SQL translation, and Administration. The main area displays the 'delivered_orders' table schema. The schema table is as follows:

Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
row	INTEGER	NULLABLE	-	-	-	-	-
customer_id	STRING	NULLABLE	-	-	-	-	-
date	DATE	NULLABLE	-	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-
order_id	INTEGER	NULLABLE	-	-	-	-	-
items	STRING	NULLABLE	-	-	-	-	-
amount	INTEGER	NULLABLE	-	-	-	-	-
delivery_time	INTEGER	NULLABLE	-	-	-	-	-
order_preparation_time	INTEGER	NULLABLE	-	-	-	-	-
premium_member	BOOLEAN	NULLABLE	-	-	-	-	-

Figure 9: Schema for delivered_orders table created in BigQuery

The screenshot shows the Google Cloud BigQuery interface. The left sidebar contains navigation options like Analysis, Data transfers, Scheduled queries, Analytics Hub, Dataform, Partner Center, Migration, Assessment, SQL translation, and Administration. The main area displays the 'other_status_orders' table schema. The schema table is as follows:

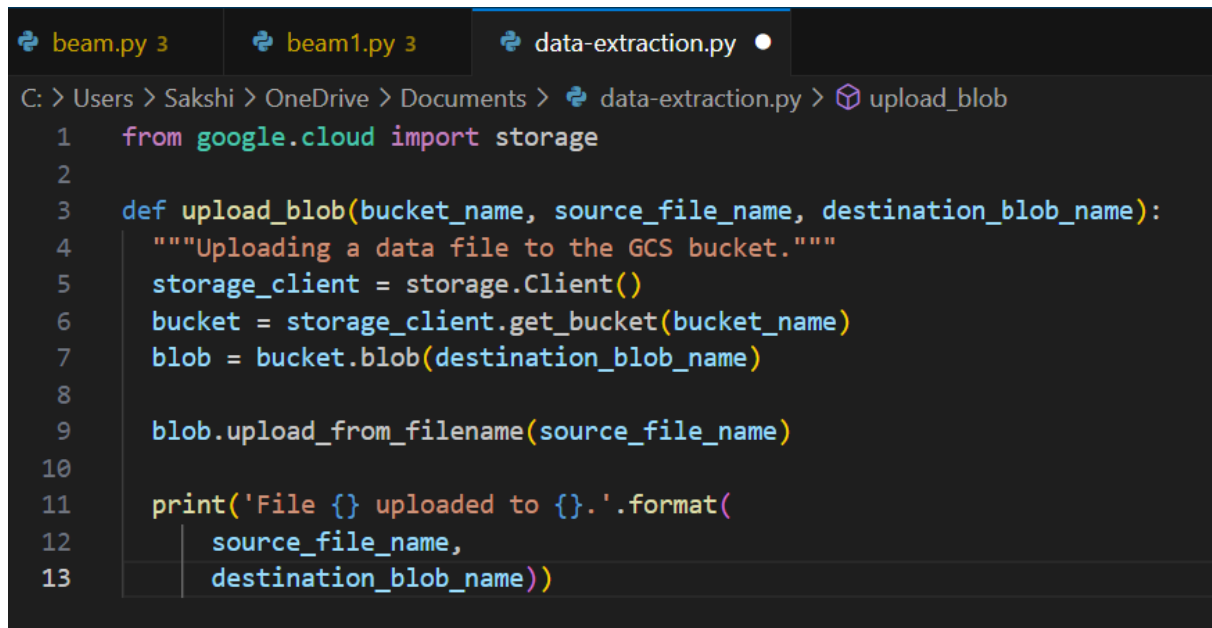
Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
row_id	INTEGER	NULLABLE	-	-	-	-	-
customer_id	INTEGER	NULLABLE	-	-	-	-	-
date	DATE	NULLABLE	-	-	-	-	-
timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-
order_id	STRING	NULLABLE	-	-	-	-	-
items	STRING	NULLABLE	-	-	-	-	-
amount	STRING	NULLABLE	-	-	-	-	-
premium_member	BOOLEAN	NULLABLE	-	-	-	-	-

Figure 10: Schema for other_status_orders table created in BigQuery

- c. Cloud Composer instance creation: Cloud Composer service will consume cloud credits hence we will create this instance during the final execution of the project. Currently this piece is being tested on Jetstream 2
- d. Dataflow: Dataflow service will consume cloud credits hence we will create this instance during the final execution of the project. Currently this piece is being tested on Jetstream 2

5.4 Data Extraction Script

Data will be extracted from the data System servers to Cloud Storage (GCS) using the following python script.



```
beam.py 3  beam1.py 3  data-extraction.py ●
C: > Users > Sakshi > OneDrive > Documents > data-extraction.py > upload_blob
1  from google.cloud import storage
2
3  def upload_blob(bucket_name, source_file_name, destination_blob_name):
4      """Uploading a data file to the GCS bucket."""
5      storage_client = storage.Client()
6      bucket = storage_client.get_bucket(bucket_name)
7      blob = bucket.blob(destination_blob_name)
8
9      blob.upload_from_filename(source_file_name)
10
11     print('File {} uploaded to {}'.format(
12         source_file_name,
13         destination_blob_name))
```

5.5 Apache Beam Pipeline proposed for Data Transformation

Apache Beam is an open-source unified programming model designed for both batch and stream processing of large-scale data sets. It provides a portable and expressive API for defining data processing pipelines that can run on various distributed processing backends, including Apache Spark, Apache Flink, Google Cloud Dataflow, and others.

Key features of Apache Beam:

- 1. Unified Model:** Apache Beam has a unified programming model for both batch and stream processing, which allows writing data processing logic once and executing it in both batch and streaming modes without modifying the code.
- 2. Portability:** Apache Beam provides a portable API that abstracts away the underlying execution engine, enabling users to run the same pipeline on different distributed processing backends.
- 3. Expressiveness:** Apache Beam offers a high-level, expressive API for defining complex data processing pipelines using familiar programming constructs such as map, filter, join, and aggregate operations.

4. Scalability: Apache Beam is designed to scale efficiently across distributed computing clusters, enabling the processing of large-scale data sets with high throughput and low latency.

5. Fault Tolerance: Apache Beam provides built-in support for fault tolerance and data durability, ensuring that data processing pipelines can recover from failures and resume processing without data loss.

6. Ecosystem Integration: Apache Beam integrates with various data storage systems, messaging platforms, and analytics tools, allowing seamless integration with existing data infrastructure.

Hence we're using Apache Beam pipeline for our data extraction and transformation process and we will proceed with testing this pipeline on jetstream2 engine.

5.6 Proposed transformations:

Data Cleaning: Cleaning the data using by removing special characters, colons and deduplicating the records ingested from the Storage bucket using beam FILTER and DISTINCT beam functions

Data Transformation: We performed normalization on certain columns like rating to ensure it is in a standard range. Additionally, we performed standardization on some columns like date with datetime data type. Aggregation was performed using GroupByKey, Combine, ParDo functions to get the summarized information.

Data Validation and Error Handling:

We will perform data validation and error handling which involves verifying the integrity, accuracy, and consistency of the final dataset that will be loaded into the data warehouse. By validating the data against predefined rules and handling errors effectively, reliability of the analyses and decision-making processes can be improved.

5.7 Testing of the Data Pipeline on jetstream2 VM

We tested the above proposed pipeline on Jetstream2 m3.quad instance by installing necessary python and apache-beam libraries and modules.

```
exouser@ashu: ~/data
exouser@ashu:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
exouser@ashu:~$ python3 --version
Python 3.10.12
exouser@ashu:~$ pip3 --version
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python 3.10)
exouser@ashu:~$ pip install apache-beam
Defaulting to user installation because normal site-packages is not writeable
Collecting apache-beam
  Downloading apache_beam-2.54.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (14.8 MB)
    14.8/14.8 MB 103.9 MB/s eta 0:00:00
Collecting typing-extensions>=3.7.0
  Downloading typing_extensions-4.10.0-py3-none-any.whl (33 kB)
Collecting pymongo<5.0.0,>=3.8.0
  Downloading pymongo-4.6.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (677 kB)
    677.2/677.2 KB 79.9 MB/s eta 0:00:00
Collecting pyarrow<15.0.0,>=3.0.0
  Downloading pyarrow-14.0.2-cp310-cp310-manylinux_2_28_x86_64.whl (38.0 MB)
    38.0/38.0 MB 46.3 MB/s eta 0:00:00
Collecting crcmod<2.0,>=1.7
  Downloading crcmod-1.7.tar.gz (89 kB)
    89.7/89.7 KB 16.2 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Collecting zstandard<1,>=0.18.0
  Downloading zstandard-0.22.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (5.4 MB)
    5.4/5.4 MB 46.1 MB/s eta 0:00:00
Requirement already satisfied: python-dateutil<3,>=2.8.0 in /usr/lib/python3/dist-packages (from apache-beam) (2.8.1)
Collecting pyarrow-hotfix<1
  Downloading pyarrow_hotfix-0.6-py3-none-any.whl (7.9 kB)
Collecting jsonpickle<4.0.0,>=3.0.0
  Downloading jsonpickle-3.0.3-py3-none-any.whl (40 kB)
    40.8/40.8 KB 10.6 MB/s eta 0:00:00
Collecting js2py<1,>=0.74
  Downloading Js2Py-0.74-py3-none-any.whl (1.0 MB)
    1.0/1.0 MB 66.2 MB/s eta 0:00:00
Collecting dill<0.3.2,>=0.3.1.1
  Downloading dill-0.3.1.1.tar.gz (111 kB)
    111.0/111.0 KB 10.6 MB/s eta 0:00:00
Installing collected packages: typing-extensions, pyarrow-hotfix, js2py, jsonpickle, dill, zstandard, pyarrow, pymongo, apache-beam
Successfully installed apache-beam-2.54.0 dill-0.3.1.1 js2py-0.74 jsonpickle-3.0.3 pyarrow-14.0.2 pyarrow-hotfix-0.6 pymongo-4.6.2 typing-extensions-4.10.0 zstandard-0.22.0
```

Apache Beam Pipeline:

```
exouser@ashu: ~/data
exouser@ashu:~/data$ cat beam.py

import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
import argparse
#from google.cloud import bigquery

#Command-line argument parser
parser = argparse.ArgumentParser()

parser.add_argument('--input',
                    dest='input',
                    required=True,
                    help='Input file to process.')

path_args, pipeline_args = parser.parse_known_args()

inputs_pattern = path_args.input

#Pipeline options
options = PipelineOptions(pipeline_args)
p = beam.Pipeline(options = options)

#Transformation functions
def remove_last_colon(row):
    #Remove the trailing colon from the fifth column of the row
    cols = row.split(',')
    item = str(cols[4])

    if item.endswith(':'):
        cols[4] = item[:-1]

    return ','.join(cols)

def remove_special_characters(row):
    #Remove special characters from each column of the row
    import re
    cols = row.split(',')
    ret = ''
    for col in cols:
        clean_col = re.sub(r'[%&]', '', col)
        ret = ret + clean_col + ','
    ret = ret[:-1]
    return ret

def print_row(row):
    print (row)
```

```
....
WARNING: The script jsonschema is installed in '/home/exouser/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-locati
on.
Successfully installed apache-beam-2.54.0 attrs-23.2.0 cloudpickle-2.2.1 crcmod-1.7 dill-0.3.1.1 dnspython-2.6.1
docopt-0.6.2 fastavro-1.9.4 fasteners-0.19 grpcio-1.62.1 hdfs-2.7.3 js2py-0.74 jsonpickle-3.0.3 jsonschema-4.21
.1 jsonschema-specifications-2023.12.1 numpy-1.24.4 objsize-0.7.0 orjson-3.9.15 packaging-24.0 proto-plus-1.23.6
protobuf-4.25.3 pyarrow-14.0.2 pyarrow-hotfix-0.6 pydot-1.4.2 pyjsparser-2.7.1 pymongo-4.6.2 referencing-0.34.6
regex-2023.12.25 rpds-py-0.18.0 typing-extensions-4.10.0 tzlocal-5.2 zstandard-0.22.0
exouser@ashu:~$ pwd
/home/exouser
exouser@ashu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
exouser@ashu:~$ mkdir data
exouser@ashu:~$ mv Downloads/food_daily.csv .
exouser@ashu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos data food_daily.csv
exouser@ashu:~$ mv food_data.csv data/
mv: cannot stat 'food_data.csv': No such file or directory
exouser@ashu:~$ rm data
rm: cannot remove 'data': Is a directory
exouser@ashu:~$ mv /home/exouser/food_data.csv home/data/
mv: cannot stat '/home/exouser/food_data.csv': No such file or directory
exouser@ashu:~$ mv /home/exouser/food_daily.csv home/data/
mv: cannot move '/home/exouser/food_daily.csv' to 'home/data/': No such file or directory
exouser@ashu:~$ mv food_daily.csv data/
exouser@ashu:~$ cd data
exouser@ashu:~/data$ ls
food_daily.csv
exouser@ashu:~/data$ vim beam.py
exouser@ashu:~/data$ python3 beam.py --input food_daily.csv
Others count:22
Total Count:891
Delivered count:869
Success!!!
```

The pipeline is working successfully when tested with the sample subset.

6. Preliminary Observations

Key metrics identified for the dashboards

From the preliminary analysis of our dataset, we have identified key performance indicators (KPIs) that possess the potential to significantly bolster the business operations. These KPIs serve as actionable metrics, providing valuable insights and guidance for strategic decision-making. By implementing these KPIs, we aim to optimize various facets of the business, driving growth, efficiency, and ultimately, greater success.

Below are the KPIs that will be implemented on Tableau:

1. Delivered orders: Mode of Payment distribution using a pie chart
2. Restaurant Rating distribution using a bar chart
3. Distribution of all orders with status: on hold, not delivered, cancelled
4. Top 5 food items
5. Top 5 restaurants
6. Popular customers distribution: displays the customers who frequently place orders

During the following phases, through additional analysis, we will recognize further KPIs that will provide actionable insights for data-driven decision making and address possible gaps in the food industry.

Issues with Apache Beam:

While building the Apache Beam pipeline on local machine we faced Installation/execution issues pertaining to some modules like 'distutils' which slightly delayed our process of implementation. This issue was resolved by updating the setup tools for Apache Beam.

```
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
File "<frozen importlib._bootstrap>", line 1331, in _find_and_load_unlocked
File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
File "<frozen importlib._bootstrap_external>", line 995, in exec_module
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "C:\Users\Sakshi\AppData\Local\Temp\pip-build-env-8zx07zqm\overlay\Lib\site-packages\setuptools\__init__.py", line 10, in <module>
    import distutils.core
ModuleNotFoundError: No module named 'distutils'

[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
error: subprocess-exited-with-error

: pip subprocess to install build dependencies did not run successfully.
exit code: 2
-> See above for output.

note: This error originates from a subprocess, and is likely not a problem with pip.
```

Conclusion:

We have completed the Proof-of-Concept (POC) phase, working with a subset of the data. The POC was successfully implemented on a Jetstream2 instance, demonstrating its feasibility. The ETL pipeline ran smoothly without any issues. Moving forward, our next steps involve adapting our ETL scripts to handle the entire dataset with scalability on cloud (GCP).

References

- [1] Research and Development of Shared Restaurant Platform Based on CloudComputing. Saddam Fuad Hussein Al Qadami
- [2] Application Research: Big Data in Food Industry
Qi Tao,¹ Hongwei Ding,¹ Huixia Wang,² and Xiaohui Cui^{1,*}
- [3]Christian Thomsen and Torben Bach Pedersen. pygrametl: A powerfulprogramming framework for extract-transform-load programmers. InProceedings of the ACM twelfth international workshop on Data ware-housing and OLAP, pages 49–56. ACM, 2009
- [4]Michael Stonebraker, Daniel Abadi, David J DeWitt, Sam Madden, ErikPaulson, Andrew Pavlo, and Alexander Rasin. Mapreduce and paralleldbms: friends or foes? Communications of the ACM, 53(1):64–71,2010